7. GÜN • YAZILIM GELİŞTİRME (SEVİYE 2)

İleri Seviye OOP

Miras Bırakmak ve Veriyi Korumak

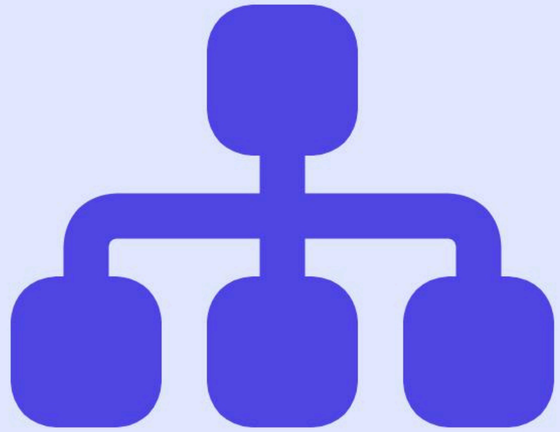
PARADİGMA DEĞİŞİMİ

Kalıtım (Inheritance) Nedir?

Daha önce aynı özellikleri defalarca yazdık.

Kalıtım ise **genetik aktarım** gibidir. Ortak özellikleri bir kez yazar, diğer sınıflara miras bırakırız.

- Kod tekrarına son verir.
- Sistemi modüler yapar.



Üst Sınıf (Parent Class)

Temel özellikleri ve genel kuralları barındıran ana şablon.



Alt Sınıf (Child Class)

Ana şablondan miras alan ve üzerine **yeni özellikler** ekleyen özel şablon.



İlk Mirasımızı Alalım

```
# 1. Üst Sınıf (Parent)
class Tasit:
def __init__(self, marka):
self.marka = marka

def bilgi(self):
print(f"Bu bir {self.marka} taşıtıdır.")

# 2. Alt Sınıf (Child) - Parantez içine Parent yazılır!
class Araba(Tasit):
pass  # Şimdilik ekstra bir şey eklemedik

# Araba, Tasit'in tüm özelliklerini otomatik kazandı
a1 = Araba("Toyota")
a1.bilgi() # Çıktı: Bu bir Toyota taşıtıdır.
```

super() Metodu

Alt sınıfa yeni bir `__init__` yazdığımızda, üst sınıfın `__init__`'i ezilir.

Bunu önlemek için `super()` kullanarak üst sınıfın özelliklerini içeri çağırırız.

Zaman ve kod kazandırır!

```
class Araba(Tasit):
    def __init__(self, marka, model):
        # Üst sınıfın init'ini çağır
        super().__init__(marka)

        # Kendi yeni özelliğimizi ekle
        self.model = model
```



Sıra Sende 1: İlk Hiyerarşi!

- `Calisan` adında bir üst sınıf oluşturun (isim ve maas alsın).
- `Yonetici` adında bir alt sınıf oluşturun (Calisan'dan miras alsın).
- Yonetici sınıfına `super()` kullanarak **departman** özelliğini ekleyin.
- Bir yönetici nesnesi üretip özelliklerini yazdırın.

Süreniz: 5 Dakika

ÇÖZÜM 1

Çalışan Sınıfı Çözümü

```
class Calisan:
    def __init__(self, isim, maas):
        self.isim = isim
        self.maas = maas

class Yonetici(Calisan):
    def __init__(self, isim, maas, departman):
        super().__init__(isim, maas) # Üst sınıfa gönderdik
        self.departman = departman   # Yeni özelliği ekledik

mudur = Yonetici("Ayşe Hanım", 45000, "IT")
print(f"Yönetici: {mudur.isim}, Departman: {mudur.departman}")
# Çıktı: Yönetici: Ayşe Hanım, Departman: IT
```

GİZLİLİK ZAMANI

Kapsülleme (Encapsulation)

Her veriye dışarıdan doğrudan müdahale edilmemelidir.

Örneğin: Bir banka hesabının **bakiyesi** dışarıdan `hesap.bakiye = 5000000` şeklinde değiştirilemez olmalıdır.

Veriyi gizleme sanatına **Kapsülleme** diyoruz.



KODLAMA VAKTİ

Private (Gizli) Değişkenler

Python'da bir değişkenin başına iki alt çizgi (__) koyarsak o değişken gizlenir.

```
class BankaHesabi:
    def __init__(self, isim, bakiye):
        self.isim = isim
        self.__bakiye = bakiye  # Private değişken (__ ile başlar)

hesap = BankaHesabi("Ali", 1000)

print(hesap.isim)       # Çıktı: Ali (Sorunsuz çalışır)
print(hesap.__bakiye)   # HATA! (AttributeError verir, gizlidir)
```

KONTROLLÜ ERİŞİM

Getter (Okuma)

Gizli veriyi sadece **okumak** için yazdığımız metotlar.



Setter (Yazma)

Gizli veriyi **kurallara bağlayarak** değiştiren metotlar.



KODLAMA VAKTİ

Getter ve Setter Uygulaması

```
class BankaHesabi:
def __init__(self, bakiye):
self.__bakiye = bakiye

# GETTER (Okuma)
def bakiye_goster(self):
return f"Güncel Bakiye: {self.__bakiye} TL"

# SETTER (Kontrollü Yazma)
def para_yatir(self, miktar):
if miktar > 0:
self.__bakiye += miktar
else:
print("Geçersiz miktar!")
```



Sıra Sende 2: Sistem Şifresi!

- `Kullanici` sınıfı oluşturun (kullanici_adi ve __sifre alsın).
- Şifreyi okumak yasak olsun! Sadece değiştirmeye (Setter) izin verin.
- `sifre_degistir(eski_sifre, yeni_sifre)` metodu yazın.
- Eğer `eski_sifre` doğruysa yeni şifreyi kaydetsin, yanlışsa hata versin.

Süreniz: 7 Dakika

ÇÖZÜM 2

Kullanıcı Sınıfı Çözümü

```
class Kullanici:
def __init__(self, ad, sifre):
self.ad = ad
self.__sifre = sifre

def sifre_degistir(self, eski_sifre, yeni_sifre):
if eski_sifre == self.__sifre:
self.__sifre = yeni_sifre
print("Şifre başarıyla güncellendi!")
else:
print("Hata: Eski şifre yanlış!")

k = Kullanici("admin", "1234")
k.sifre_degistir("0000", "5678") # Çıktı: Hata: Eski şifre yanlış!
k.sifre_degistir("1234", "5678") # Çıktı: Şifre başarıyla güncellendi!
```

AKSİYONU DEĞİŞTİR

Metot Ezme (Overriding)

Alt sınıf, üst sınıftan miras aldığı bir metodu **kendi ihtiyaçlarına göre** yeniden yazabilir.

Buna OOP dünyasında **Overriding (Ezme)** ve aynı ismin farklı şekillerde davranmasına **Polimorfizm (Çok Biçimlilik)** denir.



Davranışı Yeniden Tanımlamak

```
class Hayvan:
    def ses_cikar(self):
        print("Bu hayvanın genel bir sesi var.")

class Kedi(Hayvan):
    def ses_cikar(self):
        # Üst sınıftaki metodu tamamen ezdik!
        print("Miyav!")

class Kopek(Hayvan):
    def ses_cikar(self):
        # Kendi versiyonunu yazdı!
        print("Hav Hav!")

kedi = Kedi()
kedi.ses_cikar() # Çıktı: Miyav!
```



Sıra Sende 3: Zor Görev!

- Ana Sınıf: `Urun`. Bir adet `fiyat_hesapla()` metodu olsun ve normal fiyatı döndürsün.
- Alt Sınıf: `IndirimliUrun`. (Urun'den miras alacak)
- `IndirimliUrun` sınıfı, `fiyat_hesapla()` metodunu ezecek (Override).
- Ezilen metot, normal fiyata %20 indirim uygulayıp sonucu döndürecek.

Süreniz: 10 Dakika

Overriding Çözümü

```
class Urun:
    def __init__(self, ad, fiyat):
        self.ad = ad
        self.fiyat = fiyat

    def fiyat_hesapla(self):
        return self.fiyat

class IndirimliUrun(Urun):
    def fiyat_hesapla(self):
        # Metot ezildi. %20 İndirim formülü uygulandı.
        indirimli_hal = self.fiyat * 0.80
        return indirimli_hal

laptop = IndirimliUrun("Oyun Bilgisayarı", 50000)
print(f"Ödenecek Tutar: {laptop.fiyat_hesapla()} TL") # Çıktı: 40000.0 TL
```

SEKTÖREL BAKIŞ

Gerçek Dünyada Neden OOP?



Güvenlik

Kapsülleme sayesinde sistemin çekirdek verileri dış saldırılardan korunur.



Modülerlik

Kalıtım sayesinde projeler lego parçaları gibi birbirine eklenerek büyür.



Sürdürülebilirlik

Polimorfizm ile yeni özellikler, eski sistemi bozmadan kolayca entegre edilir.

OOP Sözlüğü

Inheritance (Kalıtım)

Alt sınıfın, üst sınıfın özelliklerini ve metotlarını miras alması.

Encapsulation (Kapsülleme)

Verilerin (__) dışarıdan doğrudan erişime kapatılması sanatı.

Overriding (Ezme)

Miras alınan bir metodun, alt sınıfta yeniden tanımlanarak değiştirilmesi.

Super() Metodu

Alt sınıfın içindeyken, üst sınıfın metotlarına (özellikle `__init__`) ulaşmayı sağlayan komut.

Günün Özeti: Neler Başardık?

- Kalıtım mantığını kurduk.
- super ( ) ile Parent sınıfa bağlandık.
- Private değişkenler ile verilerimizi gizledik.
- Getter ve Setter metotlarını yazdık.
- Overriding yaparak metotların kaderini değiştirdik!



Günün Ödevi (Mini Proje)

Oyun Karakteri Sistemi Tasarlayın:

- `Karakter` adında ana sınıf kurun (isim ve gizli __can özellikleri olsun).
- `Savasci` ve `Buyucu` adında iki alt sınıf türetin.
- `Karakter` sınıfına `saldir()` metodu ekleyin.
- Alt sınıflarda `saldir()` metodunu ezip (override) Savaşçı için "Kılıç vurdu", Büyücü için "Ateş topu attı" yazdırın.



TEBRİKLER!

Gerçek Yazılım Mimarisi (OOP) Dünyasına Hakimsiniz.

Yarın, Hata Yönetimi ve Dosya İşlemleri ile devam edeceğiz.